

Object Oriented Programming - Lecture 7 - 8

Diana Borza - diana.borza@ubbcluj.ro

April 23, 2026

"C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off." (B. Stroustrup)

- C++ standard library - STL
- Containers
- Iterators
- Lambdas

Templates I

- *Generic programming* - algorithms are written with generic types, that are going to be specified later.
- Generic programming is supported by most modern programming languages.
- Templates allow working with generic types.
- Provide a way to reuse source code. The code is written once and can then be used with many types.
- Allow defining a function or a class that operates on different kinds of types (is parametrized with different types).

Templates - the most famous program that didn't compile

- The first concrete demonstration templates of this was a program written by *Erwin Unruh*, which computed prime numbers although it did not actually finish compiling: **the list of prime numbers was part of an error message generated by the compiler on attempting to compile the code.**

[http://aszt.inf.elte.hu/~gsd/halado_cpp/ch06s04.html#
Static-metaprogramming](http://aszt.inf.elte.hu/~gsd/halado_cpp/ch06s04.html#Static-metaprogramming)

Unruh example

```
// Prime number computation by Erwin Unruh
template <int i> struct D { D(void*); operator int(); };

template <int p, int i> struct is_prime {
    enum { prim = (p%i) && is_prime<i > 2 ? p : 0, i - 1> :: prim };
};

template < int i > struct Prime_print {
    Prime_print<i-1> a;
    enum { prim = is_prime<i, i-1>::prim };
    void f() { D<i> d = prim; }
};

struct is_prime<0,0> { enum {prim=1}; };
struct is_prime<0,1> { enum {prim=1}; };
struct Prime_print<2> { enum {prim = 1}; void f() { D<2> d = prim; } };
#ifdef LAST
#define LAST 10
#endif
main () {
    Prime_print<LAST> a;
}

01 | Type 'enum{}' can't be converted to type 'D<2>' ("primes.cpp", L2/C25).
02 | Type 'enum{}' can't be converted to type 'D<3>' ("primes.cpp", L2/C25).
03 | Type 'enum{}' can't be converted to type 'D<5>' ("primes.cpp", L2/C25).
04 | Type 'enum{}' can't be converted to type 'D<7>' ("primes.cpp", L2/C25).
05 | Type 'enum{}' can't be converted to type 'D<11>' ("primes.cpp", L2/C25).
06 | Type 'enum{}' can't be converted to type 'D<13>' ("primes.cpp", L2/C25).
07 | Type 'enum{}' can't be converted to type 'D<17>' ("primes.cpp", L2/C25).
08 | Type 'enum{}' can't be converted to type 'D<19>' ("primes.cpp", L2/C25).
09 | Type 'enum{}' can't be converted to type 'D<23>' ("primes.cpp", L2/C25).
10 | Type 'enum{}' can't be converted to type 'D<29>' ("primes.cpp", L2/C25).
```

Function templates

```
template<typename T>
T maxVal(T v1, T v2){
    if(v1 > v2)
        return v1;
    return v2;
}
```

```
template<class T>
T minVal(T v1, T v2){
    if(v1 < v2)
        return v1;
    return v2;
}
```

- T is the template parameter, a type argument for the template;
- The template parameter can be introduced with any of the two keywords: `typename`, `class`.

Function templates

- The process of generating an actual function from a template function is called **instantiation**.

```
double minD = minVal<double>(4.5, 23);  
int maxI = maxVal<int>(2, 3);  
Coin c = maxVal<Coin>(Coin{10}, Coin{50}); // we must overload the comparison operators for the Coin class
```


Class templates

- A template can be seen as a skeleton or macro.
- When specific types are added to this skeleton (e.g. double), then the result is an actual C++ class.
- When instantiating a template, the compiler creates a new class with the given template argument.
- The compiler needs to have access to the implementation of the methods, to instantiate them with the template argument.
- Simple solution: **place the definition of a template in a header file.**

Templates errors - C++ is a beautiful language



mccc
@mccore111

Follow

In C++ we don't say "Missing asterisk" we say
"error C2664: 'void
std::vector<block,std::allocator<_Ty>>::push_
back(const block &)': cannot convert
argument 1 from
'std::_Vector_iterator<std::_Vector_val<std::_Si
mple_types<block>>>' to 'block &&'" and i
think that's beautiful

4:30 PM - 1 Jun 2018

292 Retweets 926 Likes



38



292



926



C++ is a beautiful language

- **A container** - is a class designed to hold and organize multiple instances of another type (either another class, or a fundamental type).
- Common operations on containers:
 - Create an empty container (via a constructor);
 - Insert a new object into the container;
 - Remove an object from the container;
 - Return the number of objects from the container;
 - Empty the container (remove all the objects from it);
 - Access to the stored objects.

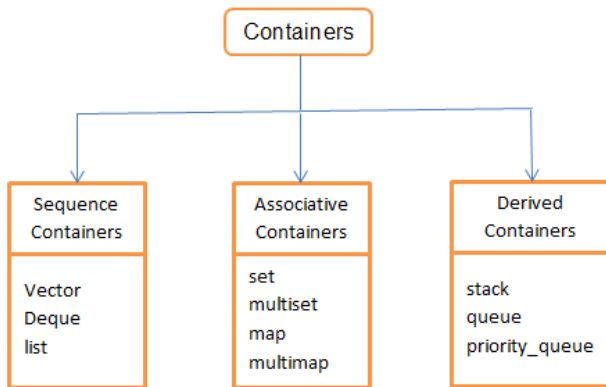
Growth rates

n	$f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10		0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20		0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30		0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40		0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50		0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100		0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000		0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000		0.013 μs	10 μs	130 μs	100 ms		
100,000		0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000		0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000		0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000		0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000		0.030 μs	1 sec	29.90 sec	31.7 years		

Figure 2.4: Growth rates of common functions measured in nanoseconds

- Advantages of using STL:
 - **simplicity**: use well written existing code, instead of writing everything from scratch;
 - **correctness**: well tested, known to be correct;
 - **efficiency**: generally, structures and algorithms from STL have a better performance than the code we write;
 - **maintainability**: code is easier to understand and more straightforward.
 - Using STL your code becomes easier to read, write, maintain and enhance!

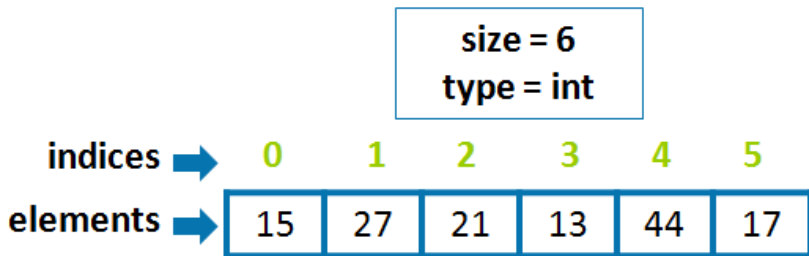
STL containers



Sequence containers

- Sequence containers are container classes that **maintain the ordering** of elements in the container.
- What is the difference between ordered and sorted?
- You can choose where to insert your element by position.

- Is a container that encapsulates fixed size arrays.
- Has the same semantics as a struct holding a C-style array $T[N]$ as its only data member.
- Combines the performance and accessibility of a C-style array with the benefits of a standard container (knowing its own size, supporting assignment, random access iterators).



- It is a sequence container that encapsulates dynamic size arrays.
- The elements are stored *contiguously* → elements can be accessed not only through iterators, but also using offsets to regular pointers to elements.
- The storage of the vector is handled automatically, being expanded and contracted as needed.
- `capacity()` actual allocated memory; `size()` - size of the vector;
- Complexity of common operations:
 - Random access (*at*, *[] operator* - constant $O(1)$;
 - Insertion or removal of elements at the end *push_back*, *pop_back* - amortized constant $O(1)$;
 - Insertion or removal of elements *insert*, *erase* - linear in the distance to the end of the vector $O(n)$

- Double-ended queue class, implemented as a dynamic array that can grow from both ends.
- Allows fast insertion and deletion at both its beginning and its end.
- As opposed to `std::vector`, the elements of a deque are not stored contiguously.
- The storage of a deque is automatically expanded and contracted as needed. Expansion of a deque is cheaper than the expansion of a `std::vector`.
- Complexity of common operations:
 - Random access - constant $O(1)$;
 - Insertion or removal of elements at the end or beginning - constant $O(1)$;
 - Insertion or removal of elements - linear $O(n)$.

- List containers are implemented as doubly-linked lists;
- Adding, removing and moving the elements does not invalidate the iterators or references. An iterator is invalidated only when the corresponding element is deleted.
- Compared to other sequence containers (array, vector and deque), lists perform generally better in *inserting*, *extracting* and *moving* elements in any position within the container;
- Main drawback: that they lack direct access to the elements by their position.

Associative containers

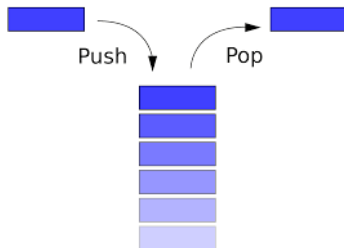
- Associative containers are containers that automatically sort their inputs when those inputs are inserted into the container. By default, associative containers compare elements using operator $<$.

- A set is a container that stores unique elements, with **duplicate elements disallowed**.
- The elements are sorted according to their values. Sorting is done using a comparison function that you can redefine.
- **A multiset** is a set where duplicate elements are allowed.

- A **map**: is a set where each element is a *key/value pair*.
- The *key* is used for *sorting* and *indexing* the data, and must be unique.
- The value is the actual data.
- A **multimap** (a dictionary) is a map that allows duplicate keys.

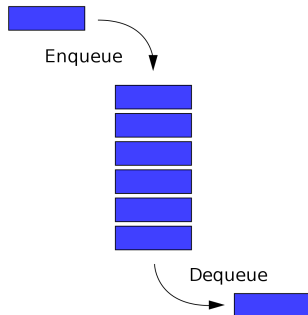
- Derived containers are special predefined containers that are adapted to specific uses.
- They provide a different interface for sequential containers.
 - stack
 - queue
 - priority_queue

- a container where elements operate in a **LIFO** (Last In, First Out) context;
- elements are inserted (pushed) to the front of the container;
- elements are removed (popped) from the front of the container;
- use deque as their default sequence container.



std::queue, std::priority_queue

- a container where elements operate in a **FIFO** (First In, First Out) context;
- elements are inserted (pushed) to the back of the container;
- elements are removed (popped) from the front of the container;
- use deque as their default sequence container.
- A **priority queue** is a queue where the elements are sorted (via operator<). At push the element is sorted in the queue. At pop (deque) the element with the highest priority is removed.



- An **Iterator** is an object that can traverse (iterate over) a container class without the user having to know how the container is implemented.
- ? What OOP feature is this ?
- Can be visualized as a pointer to a given element in the container.

- Operators defined in iterators:
 - **Operator*** : dereferences the iterator → return the element that the iterator is pointing at.
 - **Operator++**: moves the iterator to the next element in the container.
 - **Operator--** - to move to the previous element.
 - **Operator==** and **Operator!=**: basic comparison operators;
 - **Operator=**: assigns the iterator to a new position.

- All C++ containers provide (at least) two types of iterators:
 - `container::iterator` - read/write iterator
 - `container::const_iterator` - read-only iterator
- Each container includes four basic member functions to work with iterators:
 - **`begin()`**: returns an iterator representing the beginning of the elements in the container.
 - **`end()`**: returns an iterator representing the element just past the end of the elements.
 - **`cbegin()`**: returns a const (read-only) iterator representing the beginning of the elements in the container.
 - **`cend()`**: returns a const (read-only) iterator representing the element just past the end of the elements.

Lambda expressions I

- A **lambda expression** (**lambda** or **closure**) allows the definition of an anonymous function inside another function.
- The anonymous function is defined in the function where it is called.
- Very useful for some algorithms defined in the STL [std::find_if](#), [std::count_if](#), [std::transform](#).
- Syntax:

```
[ captureClause ] ( parameters ) -> returnType  
{ function body; }  
[captureClause] (parameter list) {function body;}
```

- *captureClause* and *parameters* can be empty if not needed;
- *returnType* is optional, and if omitted, [auto](#) will be assumed.

Lambda expressions II

- The **captureClause** clause is used to (indirectly) give a lambda access to variables available in the surrounding scope that it normally would not have access to.
- We need to list all the variables we want to access from within the lambda as part of the capture clause.
- For each variable in **captureClause**, a **clone!** of that variable is made (with an identical name) inside the lambda.
- A default capture (also called a capture-default) captures all variables that are mentioned in the lambda.
 - To capture all used variables by value, use a capture value of `=`.
 - To capture all used variables by reference, use a capture value of `&`.

STL algorithms

- algorithms are implemented as functions that operate using iterators'
- `std::min_element()` and `std::max_element()` algorithms find the min and max element in a container class;
- `std::find()` algorithm to finds a value in a container; returns the end of the iterator if the element is not present in the container;
- `std::sort()` - sorts a container; doesn't work on list container classes!
- `std::reverse()` - reverses a container;

Decisions, decisions...

